



# 1. Load and Understand the Dataset

We will load the dataset using pandas. we need to understand the structure of the data.

```
In [6]: import pandas as pd

# Load the dataset
df = pd.read_csv(r"C:\Users\Sai Lalit\Downloads\marketing_campaign_response_da
df.head()
```

```
Out[6]:
```

|   | customer_id | age | age_group | gender | region | membership_level | annual_incc |
|---|-------------|-----|-----------|--------|--------|------------------|-------------|
| 0 | MC0001      | 21  | 18-25     | Male   | North  | Gold             | 60          |
| 1 | MC0002      | 39  | 36-50     | Male   | South  | Basic            | 83          |
| 2 | MC0003      | 23  | 18-25     | Female | South  | Gold             | 28          |
| 3 | MC0004      | 48  | 36-50     | Female | East   | Basic            | 126         |
| 4 | MC0005      | 59  | 51+       | Female | North  | Platinum         | 112         |

```
In [7]: df.shape
```

```
Out[7]: (380, 17)
```

```
In [8]: df.columns
```

```
Out[8]: Index(['customer_id', 'age', 'age_group', 'gender', 'region',
               'membership_level', 'annual_income', 'previous_purchases',
               'average_order_value', 'website_visits_last_month',
               'emails_opened_last_month', 'ad_clicks_last_month',
               'days_since_last_purchase', 'customer_service_contacts',
               'discount_received', 'preferred_channel', 'campaign_response'],
              dtype='object')
```

```
In [9]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 380 entries, 0 to 379
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customer_id                          380 non-null    object
1   age                                  380 non-null    int64
2   age_group                            380 non-null    object
3   gender                               380 non-null    object
4   region                               380 non-null    object
5   membership_level                     380 non-null    object
6   annual_income                        380 non-null    int64
7   previous_purchases                   380 non-null    int64
8   average_order_value                  380 non-null    float64
9   website_visits_last_month            380 non-null    int64
10  emails_opened_last_month              380 non-null    int64
11  ad_clicks_last_month                  380 non-null    int64
12  days_since_last_purchase              380 non-null    int64
13  customer_service_contacts             380 non-null    int64
14  discount_received                     380 non-null    object
15  preferred_channel                     380 non-null    object
16  campaign_response                     380 non-null    int64
dtypes: float64(1), int64(9), object(7)
memory usage: 50.6+ KB

```

Missing values can effect the model performance. we need to clean the missing data in the dataset.

```
In [11]: df.isnull().sum()
```

```

Out[11]: customer_id      0
         age              0
         age_group        0
         gender           0
         region           0
         membership_level  0
         annual_income     0
         previous_purchases 0
         average_order_value 0
         website_visits_last_month 0
         emails_opened_last_month 0
         ad_clicks_last_month 0
         days_since_last_purchase 0
         customer_service_contacts 0
         discount_received  0
         preferred_channel  0
         campaign_response  0
dtype: int64

```

## Duplicate rows need to clear as well in the dataset

```
In [12]: df["campaign_response"].value_counts()
```

```
Out[12]: campaign_response
1      331
0       49
Name: count, dtype: int64
```

```
In [13]: df["campaign_response"].value_counts(normalize=True) * 100
```

```
Out[13]: campaign_response
1      87.105263
0      12.894737
Name: proportion, dtype: float64
```

In this step, we loaded the dataset and explored its basic structure. We checked the dataset size, column names, data types, missing values, duplicate rows, and the distribution of the target variable.

This step is important because machine learning models should not be trained before understanding the data.

## 2. Train/Test the dataset

In this step, we divide the dataset into two parts: training data and testing data. The training data is used to teach the machine learning model, and the testing data is used to evaluate how well the model works on new data.

```
In [14]: y = df["campaign_response"]

X = df.drop(
    columns=[
        "customer_id",
        "campaign_response"
    ]
)

X.head()
```

```
Out[14]:
```

|   | age | age_group | gender | region | membership_level | annual_income | previous |
|---|-----|-----------|--------|--------|------------------|---------------|----------|
| 0 | 21  | 18-25     | Male   | North  | Gold             | 60937         |          |
| 1 | 39  | 36-50     | Male   | South  | Basic            | 83183         |          |
| 2 | 23  | 18-25     | Female | South  | Gold             | 28783         |          |
| 3 | 48  | 36-50     | Female | East   | Basic            | 126830        |          |
| 4 | 59  | 51+       | Female | North  | Platinum         | 112255        |          |

```
In [15]: fairness_columns = df[
    [
        "gender",
        "age_group",
        "region"
    ]
]

X = X.drop(
    columns=[
        "gender",
        "age_group",
        "region"
    ]
)

X.head()
```

```
Out[15]:
```

|   | age | membership_level | annual_income | previous_purchases | average_order_v |
|---|-----|------------------|---------------|--------------------|-----------------|
| 0 | 21  | Gold             | 60937         | 6                  | 35              |
| 1 | 39  | Basic            | 83183         | 5                  | 37              |
| 2 | 23  | Gold             | 28783         | 18                 | 38              |
| 3 | 48  | Basic            | 126830        | 32                 | 19              |
| 4 | 59  | Platinum         | 112255        | 16                 | 9               |

```
In [16]: # Converting Categorical variables.

X = pd.get_dummies(
    X,
    drop_first=True)

X.head()
```

Out[16]:

|   | age | annual_income | previous_purchases | average_order_value | website_visits |
|---|-----|---------------|--------------------|---------------------|----------------|
| 0 | 21  | 60937         | 6                  | 356.21              |                |
| 1 | 39  | 83183         | 5                  | 372.06              |                |
| 2 | 23  | 28783         | 18                 | 384.90              |                |
| 3 | 48  | 126830        | 32                 | 196.58              |                |
| 4 | 59  | 112255        | 16                 | 92.20               |                |

In [17]:

```
# Train/ test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test, fairness_train, fairness_test = train_test_split(
    X,
    y,
    fairness_columns,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

(304, 16)  
(76, 16)

### 3. Train the model

In [18]:

```
# Import DecisionTreeClassifier from scikit-learn
# This model is used for classification problems

from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=4,
    random_state=42)

model.fit(
    X_train, y_train)

y_pred = model.predict(X_test)

# Create a small DataFrame to compare actual and predicted values
# y_test contains the real churn values
# y_pred contains the model predictions
comparison = pd.DataFrame({
```

```

    "Actual Churn": y_test.values,
    "Predicted Churn": y_pred
})

# Display the first 10 rows of the comparison table
comparison.head(10)

```

Out[18]:

|   | Actual Churn | Predicted Churn |
|---|--------------|-----------------|
| 0 | 1            | 1               |
| 1 | 1            | 1               |
| 2 | 1            | 0               |
| 3 | 1            | 1               |
| 4 | 1            | 1               |
| 5 | 1            | 1               |
| 6 | 1            | 1               |
| 7 | 1            | 1               |
| 8 | 1            | 1               |
| 9 | 1            | 1               |

This table compares the real churn values with the values predicted by the Decision Tree model. The column Actual Churn shows the real customer status, and the column Predicted Churn shows the model prediction.

In this dataset, 0 means the customer did not churn, and 1 means the customer churned.

## 4. Evaluate the model

In this step, we evaluate how well the trained model performs. We compare the model performance on the training data and the testing data.

The training score shows how well the model learned from the training data. The testing score shows how well the model works on new and unseen data.

This step is important because a model may perform very well on training data but poorly on testing data. This problem is called **overfitting**.

```

In [19]: # Import accuracy_score to calculate the accuracy of the model
          from sklearn.metrics import accuracy_score

          # Predict churn values for the training data

```

```

y_train_pred = model.predict(X_train)

# Predict churn values for the testing data
y_test_pred = model.predict(X_test)

# Calculate the accuracy score for the training data
train_accuracy = accuracy_score(y_train, y_train_pred)

# Calculate the accuracy score for the testing data
test_accuracy = accuracy_score(y_test, y_test_pred)

# Create a DataFrame to show the evaluation results clearly
evaluation_results = pd.DataFrame({
    "Dataset": ["Training Data", "Testing Data"],
    "Accuracy": [train_accuracy, test_accuracy]
})

# Display the evaluation results
evaluation_results

```

```

Out[19]:

```

|   | Dataset       | Accuracy |
|---|---------------|----------|
| 0 | Training Data | 0.957237 |
| 1 | Testing Data  | 0.815789 |

Analysis of Training and Testing Accuracy The model achieved a training accuracy of 0.9572, which means that it correctly predicted about 95.72% of the training data. This shows that the model learned some patterns from the training dataset.

However, the testing accuracy is 0.8157, which means that the model correctly predicted only about 81.57% of the unseen test data. This result is much lower than the training accuracy.

The difference between training accuracy and testing accuracy is large. The model performs better on the training data than on the testing data. This suggests that the model may be experiencing **overfitting**.

## 5. Accuracy, Precision, Recall, and F1-Score

In this step, we evaluate the model using four common classification metrics: accuracy, precision, recall, and F1-score.

These metrics help us understand the model performance more clearly than accuracy alone. This is especially important in business problems such as customer churn prediction.

```
In [20]: # Import evaluation metrics from scikit-learn
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Use the trained model to make predictions on the test data
y_test_pred = model.predict(X_test)

# Calculate accuracy
# Accuracy shows the percentage of all correct predictions
accuracy = accuracy_score(y_test, y_test_pred)

# Calculate precision
# Precision shows how many predicted churn customers were actually churn customers
precision = precision_score(y_test, y_test_pred, zero_division=0)

# Calculate recall
# Recall shows how many actual churn customers were correctly identified by the model
recall = recall_score(y_test, y_test_pred, zero_division=0)

# Calculate F1-score
# F1-score is the balance between precision and recall
f1 = f1_score(y_test, y_test_pred, zero_division=0)

# Create a table to display the evaluation metrics
metrics_results = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Score": [accuracy, precision, recall, f1]
})

# Display the metrics table
metrics_results
```

```
Out[20]:
```

|          | <b>Metric</b> | <b>Score</b> |
|----------|---------------|--------------|
| <b>0</b> | Accuracy      | 0.815789     |
| <b>1</b> | Precision     | 0.871429     |
| <b>2</b> | Recall        | 0.924242     |
| <b>3</b> | F1-Score      | 0.897059     |

Analysis of Accuracy, Precision, Recall, and F1-Score The model has an accuracy of 0.8157, which means that it correctly predicted about 81.57% of the test data. This result is relatively weak, because the model is only slightly better than random guessing.

The precision score is 0.8714. This means that when the model predicted that a customer would churn, it was correct about 87.14% of the time. In other words, many customers predicted as churn may not actually churn.

The recall score is 0.9242. This means that the model correctly identified about

92.42% of the customers who actually churned. Compared with precision, recall is higher, so the model is better at finding churn customers than avoiding false churn predictions.

The F1-score is 0.8970. The F1-score combines precision and recall into one balanced score. Since this value is high.

### Meaning of the Evaluation Metrics

Accuracy: shows the overall percentage of correct predictions. It answers this question: How many predictions were correct in total?

Precision: focuses on customers predicted as churn. It answers this question: Of all customers predicted as churn, how many actually churned?

Recall: focuses on actual churn customers. It answers this question: Of all customers who actually churned, how many did the model correctly find?

F1-score: combines precision and recall into one score. It is useful when we want a balance between finding churn customers and avoiding wrong churn predictions.

## 6. Confusion Matrix

```
In [22]: # Import confusion_matrix to calculate the confusion matrix
from sklearn.metrics import confusion_matrix

# Import ConfusionMatrixDisplay to visualize the confusion matrix
from sklearn.metrics import ConfusionMatrixDisplay

# Import matplotlib to display the plot
import matplotlib.pyplot as plt

# Use the trained model to predict churn values for the test data
y_test_pred = model.predict(X_test)

# Create the confusion matrix by comparing actual values and predicted values
cm = confusion_matrix(y_test, y_test_pred)

# Extract the four values from the confusion matrix
tn, fp, fn, tp = cm.ravel()

# Create a DataFrame to show the confusion matrix in table format
cm_table = pd.DataFrame(
    cm,
    index=["Actual Not Churn (0)", "Actual Churn (1)"],
    columns=["Predicted Not Churn (0)", "Predicted Churn (1)"])

# Display the confusion matrix table
```

cm\_table

Out[22]:

|                      | Predicted Not Churn (0) | Predicted Churn (1) |
|----------------------|-------------------------|---------------------|
| Actual Not Churn (0) | 1                       | 9                   |
| Actual Churn (1)     | 5                       | 61                  |

Confusion Matrix Results True Negative = 1: The model correctly predicted that 1 customers would not churn.

False Positive = 9: The model predicted that 9 customers would churn, but they actually did not churn.

False Negative = 5: The model predicted that 5 customers would not churn, but they actually churned.

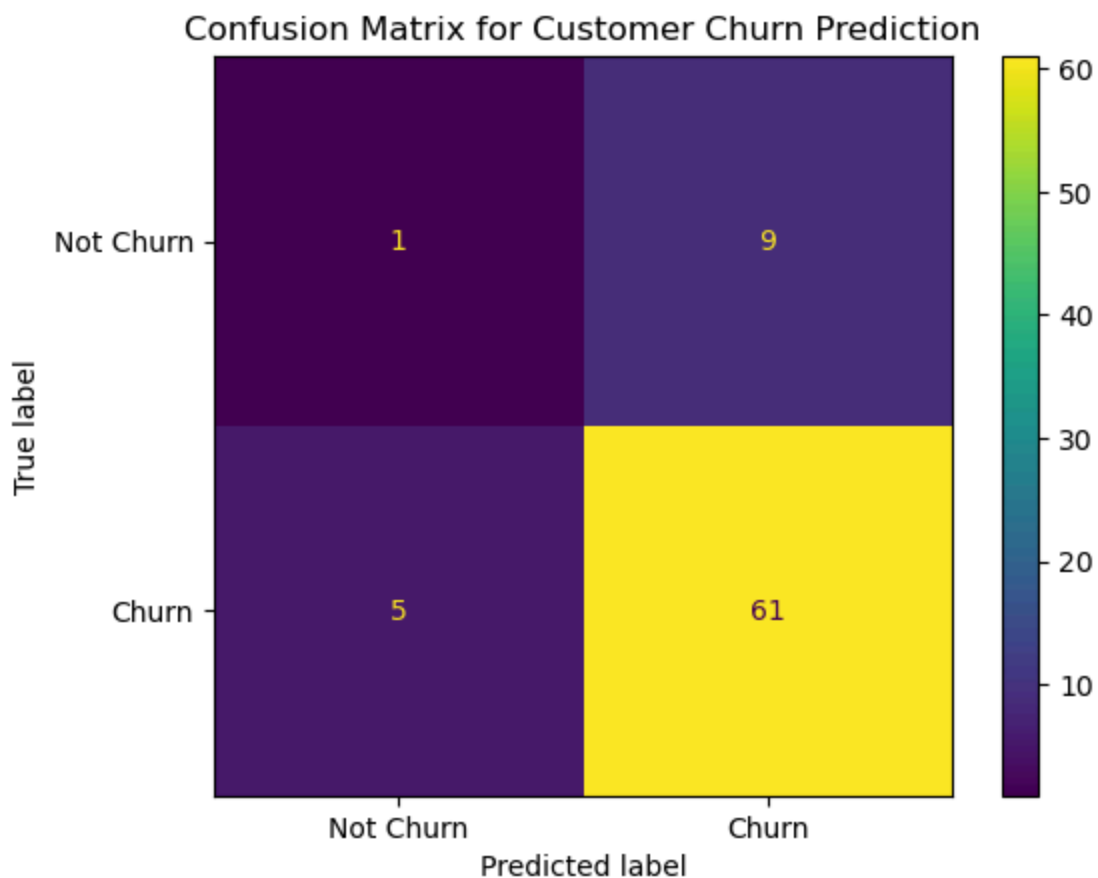
True Positive = 61: The model correctly predicted that 61 customers would churn.

```
In [24]: # Create a visual display of the confusion matrix
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Churn", "Churn"]
)

# Plot the confusion matrix
display.plot()

# Add a title to the plot
plt.title("Confusion Matrix for Customer Churn Prediction")

# Show the plot
plt.show()
```



### Conclusion

Overall, the confusion matrix shows that the model can identify some churn customers, but it still makes many mistakes. The model is especially weak in predicting Not Churn customers correctly. Therefore, this model should be improved before being used for real customer retention decisions.

```
In [25]: # Creating a summary table for the four confusion matrix values

confusion_summary = pd.DataFrame({
    "Type": ["True Negative", "False Positive", "False Negative", "True Positive"],
    "Meaning": [
        "The model correctly predicted that the customer did not churn",
        "The model predicted churn, but the customer did not churn",
        "The model predicted not churn, but the customer actually churned",
        "The model correctly predicted that the customer churned"
    ],
    "Value": [tn, fp, fn, tp]
})

confusion_summary
```

Out[25]:

|   | Type           | Meaning                                           | Value |
|---|----------------|---------------------------------------------------|-------|
| 0 | True Negative  | The model correctly predicted that the custome... | 1     |
| 1 | False Positive | The model predicted churn, but the customer di... | 9     |
| 2 | False Negative | The model predicted not churn, but the custome... | 5     |
| 3 | True Positive  | The model correctly predicted that the custome... | 61    |

## 7. Error Analysis

In this step, we analyze the model errors more carefully. Instead of only looking at overall accuracy, we check which predictions were correct and which predictions were wrong.

Error analysis helps us understand the business meaning of model mistakes. For customer churn prediction, the two important errors are **false positives** and **false negatives**.

```
In [26]: y_test_pred = model.predict(X_test)

# Create a new DataFrame to compare actual and predicted churn values
error_analysis = pd.DataFrame({
    "Customer ID": df.loc[y_test.index, "customer_id"].values,
    "Actual Churn": y_test.values,
    "Predicted Churn": y_test_pred
})

# Create a new column to check whether each prediction is correct
error_analysis["Correct Prediction"] = error_analysis["Actual Churn"] == error_analysis["Predicted Churn"]

# Display the first 10 rows of the error analysis table
error_analysis.head(10)
```

Out[26]:

|   | Customer ID | Actual Churn | Predicted Churn | Correct Prediction |
|---|-------------|--------------|-----------------|--------------------|
| 0 | MC0362      | 1            | 1               | True               |
| 1 | MC0049      | 1            | 1               | True               |
| 2 | MC0036      | 1            | 0               | False              |
| 3 | MC0055      | 1            | 1               | True               |
| 4 | MC0304      | 1            | 1               | True               |
| 5 | MC0029      | 1            | 1               | True               |
| 6 | MC0088      | 1            | 1               | True               |
| 7 | MC0230      | 1            | 1               | True               |
| 8 | MC0176      | 1            | 1               | True               |
| 9 | MC0147      | 1            | 1               | True               |

### Classify Prediction Results

Now we classify each prediction into one of four groups: True Negative, False Positive, False Negative, or True Positive. This helps us understand the type of each prediction.

```
In [27]: def classify_error(row):

    # True Negative: actual value is 0 and predicted value is 0
    if row["Actual Churn"] == 0 and row["Predicted Churn"] == 0:
        return "True Negative"

    # False Positive: actual value is 0 but predicted value is 1
    elif row["Actual Churn"] == 0 and row["Predicted Churn"] == 1:
        return "False Positive"

    # False Negative: actual value is 1 but predicted value is 0
    elif row["Actual Churn"] == 1 and row["Predicted Churn"] == 0:
        return "False Negative"

    # True Positive: actual value is 1 and predicted value is 1
    elif row["Actual Churn"] == 1 and row["Predicted Churn"] == 1:
        return "True Positive"

    # Apply the function to each row and create a new column called Error Type
    error_analysis["Error Type"] = error_analysis.apply(classify_error, axis=1)

    # Display the first 10 rows with the error type
    error_analysis.head(10)
```

Out[27]:

|   | Customer ID | Actual Churn | Predicted Churn | Correct Prediction | Error Type     |
|---|-------------|--------------|-----------------|--------------------|----------------|
| 0 | MC0362      | 1            | 1               | True               | True Positive  |
| 1 | MC0049      | 1            | 1               | True               | True Positive  |
| 2 | MC0036      | 1            | 0               | False              | False Negative |
| 3 | MC0055      | 1            | 1               | True               | True Positive  |
| 4 | MC0304      | 1            | 1               | True               | True Positive  |
| 5 | MC0029      | 1            | 1               | True               | True Positive  |
| 6 | MC0088      | 1            | 1               | True               | True Positive  |
| 7 | MC0230      | 1            | 1               | True               | True Positive  |
| 8 | MC0176      | 1            | 1               | True               | True Positive  |
| 9 | MC0147      | 1            | 1               | True               | True Positive  |

In [28]:

```
# Count how many times each error type appears
error_counts = error_analysis["Error Type"].value_counts().reset_index()

# Rename the columns for better readability
error_counts.columns = ["Error Type", "Count"]

# Display the error count table
error_counts
```

Out[28]:

|   | Error Type     | Count |
|---|----------------|-------|
| 0 | True Positive  | 61    |
| 1 | False Positive | 9     |
| 2 | False Negative | 5     |
| 3 | True Negative  | 1     |

In [29]:

```
# Select only the rows where the model prediction was wrong
wrong_predictions = error_analysis[error_analysis["Correct Prediction"] == False]

# Display the first 10 wrong predictions
wrong_predictions.head(10)
```

Out[29]:

|    | Customer ID | Actual Churn | Predicted Churn | Correct Prediction | Error Type     |
|----|-------------|--------------|-----------------|--------------------|----------------|
| 2  | MC0036      | 1            | 0               | False              | False Negative |
| 12 | MC0045      | 1            | 0               | False              | False Negative |
| 14 | MC0112      | 1            | 0               | False              | False Negative |
| 17 | MC0354      | 0            | 1               | False              | False Positive |
| 19 | MC0234      | 0            | 1               | False              | False Positive |
| 20 | MC0128      | 0            | 1               | False              | False Positive |
| 23 | MC0380      | 0            | 1               | False              | False Positive |
| 29 | MC0273      | 0            | 1               | False              | False Positive |
| 36 | MC0236      | 1            | 0               | False              | False Negative |
| 48 | MC0110      | 0            | 1               | False              | False Positive |

## Conclusion

This table helps us understand the model's mistakes at the customer level. The model seems to predict churn too often, which creates many false positives. To improve the model, we should reduce both false positives and false negatives, especially if the business wants more reliable customer retention decisions.

```
In [30]: import matplotlib.pyplot as plt

# Create a simple bar chart for error types
error_counts.plot(
    x="Error Type",
    y="Count",
    kind="bar",
    legend=False
)

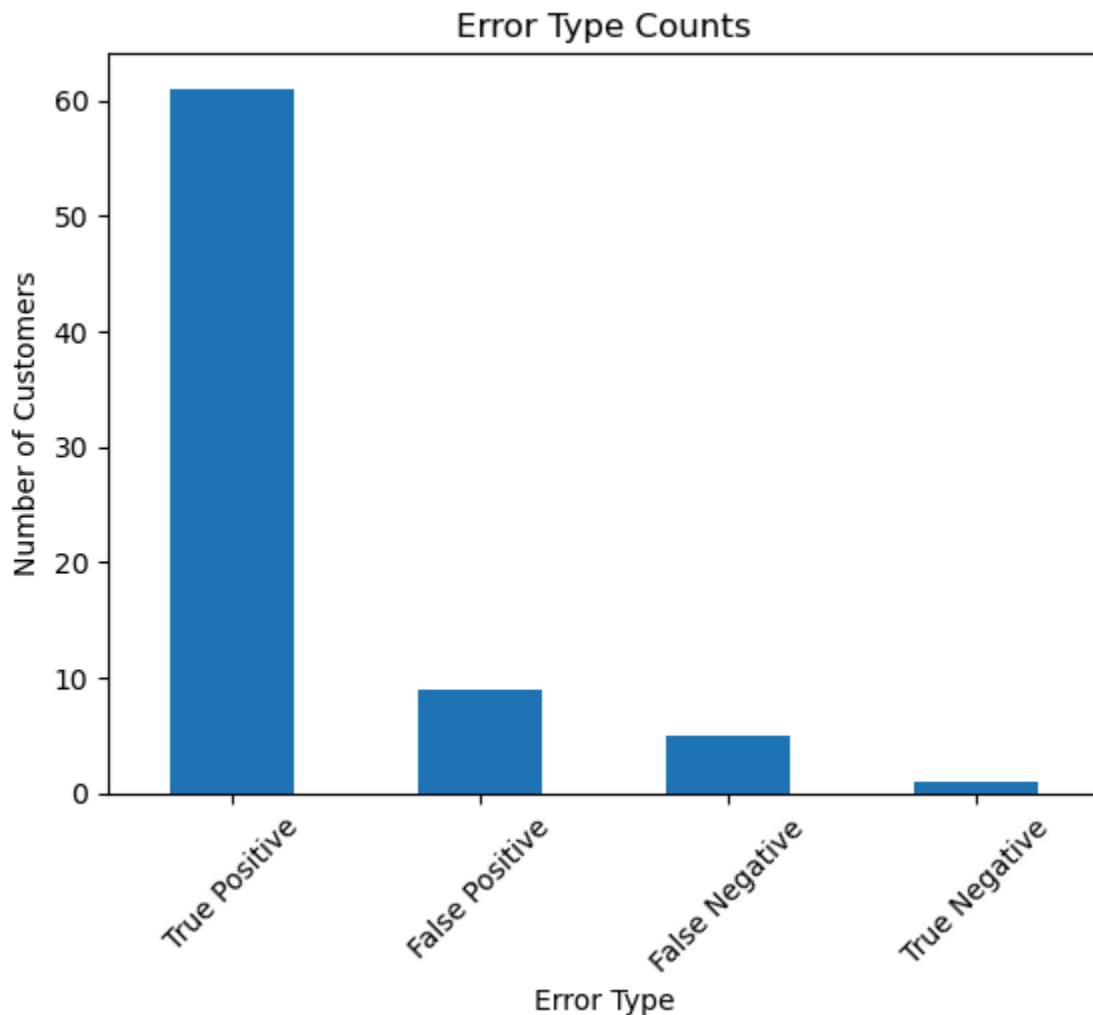
# Add a title to the chart
plt.title("Error Type Counts")

# Add a label for the x-axis
plt.xlabel("Error Type")

# Add a label for the y-axis
plt.ylabel("Number of Customers")

# Rotate x-axis labels for better readability
plt.xticks(rotation=45)
```

```
# Show the chart  
plt.show()
```



## 8. Cross Validation

In this step, we use cross-validation to evaluate the model more reliably. Instead of testing the model only once, cross-validation trains and tests the model several times on different parts of the dataset.

This helps us understand whether the model performance is stable or depends too much on one specific train/test split.

```
In [31]: from sklearn.model_selection import cross_val_score  
  
# Import StratifiedKFold to keep the same target distribution in each fold  
from sklearn.model_selection import StratifiedKFold  
  
# Create a 5-fold cross-validation strategy
```

```

# n_splits=5 means the dataset will be divided into 5 parts
# shuffle=True means the data will be shuffled before splitting
# random_state=42 makes the result reproducible
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Apply cross-validation to the Decision Tree model
# X contains the input features
# y contains the target variable
# scoring="accuracy" means we evaluate the model using accuracy
cv_scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

# Create a table to show the accuracy score for each fold
cv_results = pd.DataFrame({
    "Fold": ["Fold 1", "Fold 2", "Fold 3", "Fold 4", "Fold 5"],
    "Accuracy": cv_scores
})

# Display the cross-validation results
cv_results

```

Out[31]:

|   | Fold   | Accuracy |
|---|--------|----------|
| 0 | Fold 1 | 0.868421 |
| 1 | Fold 2 | 0.855263 |
| 2 | Fold 3 | 0.842105 |
| 3 | Fold 4 | 0.868421 |
| 4 | Fold 5 | 0.815789 |

In [32]:

```

# Calculate the average accuracy across all folds
mean_cv_accuracy = cv_scores.mean()

# Calculate the standard deviation of accuracy across all folds
std_cv_accuracy = cv_scores.std()

# Create a summary table for cross-validation results
cv_summary = pd.DataFrame({
    "Metric": ["Mean Cross-Validation Accuracy", "Standard Deviation"],
    "Value": [mean_cv_accuracy, std_cv_accuracy]
})

# Display the summary table
cv_summary

```

Out[32]:

|   | Metric                         | Value    |
|---|--------------------------------|----------|
| 0 | Mean Cross-Validation Accuracy | 0.850000 |
| 1 | Standard Deviation             | 0.019693 |

In [33]:

```

# Import matplotlib for visualization

```

```
import matplotlib.pyplot as plt

# Create a bar chart for cross-validation accuracy scores
cv_results.plot(
    x="Fold",
    y="Accuracy",
    kind="bar",
    legend=False
)

# Add a title to the chart
plt.title("Cross-Validation Accuracy Scores")

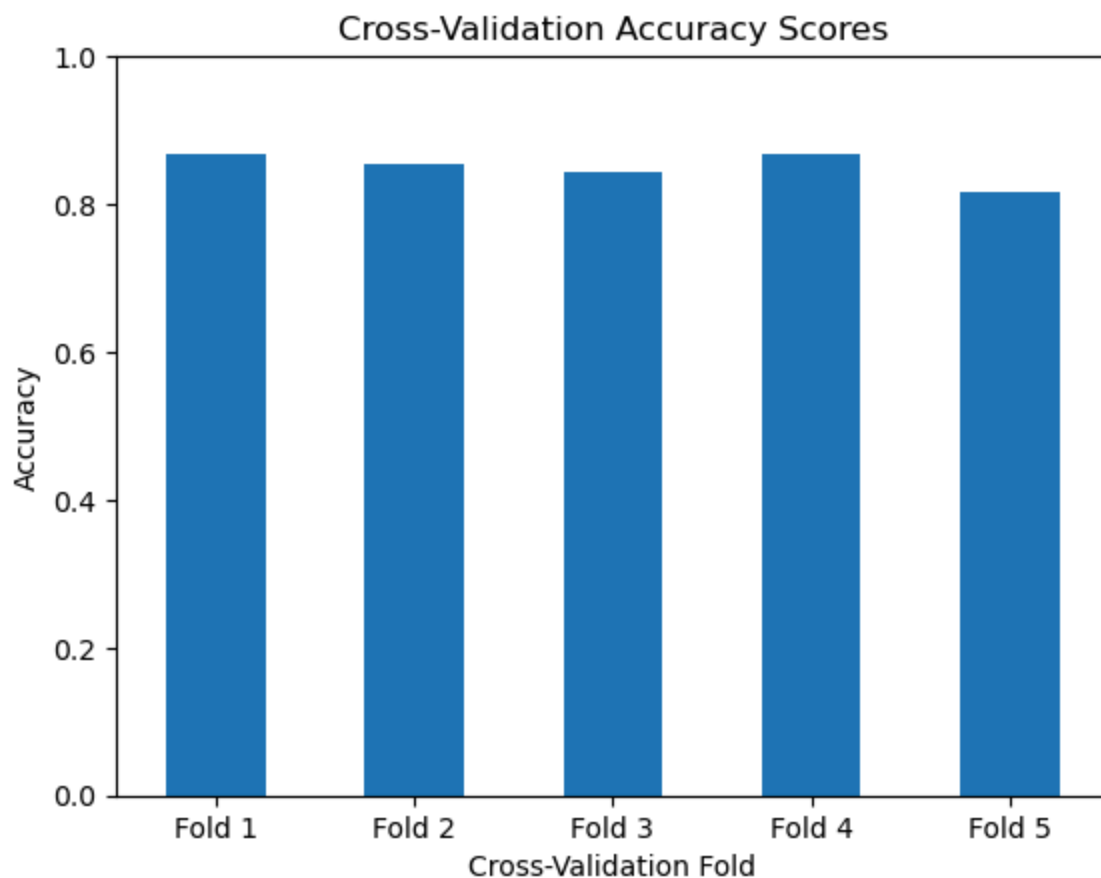
# Add a label for the x-axis
plt.xlabel("Cross-Validation Fold")

# Add a label for the y-axis
plt.ylabel("Accuracy")

# Set the y-axis range from 0 to 1
plt.ylim(0, 1)

# Rotate x-axis labels for better readability
plt.xticks(rotation=0)

# Show the chart
plt.show()
```



## 9. Overfitting and Underfitting

In this step, we study two common machine learning problems: overfitting and underfitting. These problems help us understand whether the model is too simple, too complex, or balanced.

Underfitting happens when the model is too simple and cannot learn enough patterns from the training data. In this case, both training accuracy and testing accuracy are usually low.

Overfitting happens when the model learns the training data too specifically. In this case, training accuracy is high, but testing accuracy is much lower.

```
In [37]: import pandas as pd

# Import train_test_split to split the dataset into training and testing sets
from sklearn.model_selection import train_test_split

# Import DecisionTreeClassifier to build the machine learning model
from sklearn.tree import DecisionTreeClassifier

# Import accuracy_score to evaluate the model performance
```

```

from sklearn.metrics import accuracy_score

# Load the CSV dataset
df = pd.read_csv(r"C:\Users\Sai Lalit\Downloads\marketing_campaign_response_da

# Define the target variable
y = df["campaign_response"]

# Define the input features and remove customer_id and churn
X = df.drop(columns=["customer_id", "campaign_response"])

# Remove fairness-related columns from the model input
X = X.drop(columns=["gender", "age_group", "region"])

# Convert categorical variables into numerical variables
X = pd.get_dummies(X, drop_first=True)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create a Decision Tree model
model = DecisionTreeClassifier(max_depth=4, random_state=42)

# Train the model using the training data
model.fit(X_train, y_train)

# Predict churn values for the training data
y_train_pred = model.predict(X_train)

# Predict churn values for the testing data
y_test_pred = model.predict(X_test)

# Calculate training accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Calculate testing accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

# Create a table to compare training and testing accuracy
overfitting_check = pd.DataFrame({
    "Dataset": ["Training Data", "Testing Data"],
    "Accuracy": [train_accuracy, test_accuracy]
})

# Display the comparison table
overfitting_check

```

Out[37]:

|   | <b>Dataset</b> | <b>Accuracy</b> |
|---|----------------|-----------------|
| 0 | Training Data  | 0.957237        |
| 1 | Testing Data   | 0.815789        |

|   |               |          |
|---|---------------|----------|
| 0 | Training Data | 0.957237 |
| 1 | Testing Data  | 0.815789 |

```
In [38]: # Import DecisionTreeClassifier to create decision tree models
from sklearn.tree import DecisionTreeClassifier

# Create an empty list to store the results
depth_results = []

# Test different max_depth values from 1 to 10
for depth in range(1, 11):

    # Create a Decision Tree model with the current depth
    temp_model = DecisionTreeClassifier(max_depth=depth, random_state=42)

    # Train the model using the training data
    temp_model.fit(X_train, y_train)

    # Predict values for the training data
    train_pred = temp_model.predict(X_train)

    # Predict values for the testing data
    test_pred = temp_model.predict(X_test)

    # Calculate training accuracy
    train_acc = accuracy_score(y_train, train_pred)

    # Calculate testing accuracy
    test_acc = accuracy_score(y_test, test_pred)

    # Save the depth, training accuracy, and testing accuracy
    depth_results.append([depth, train_acc, test_acc])

# Convert the results into a DataFrame
depth_results_df = pd.DataFrame(
    depth_results,
    columns=["Max Depth", "Training Accuracy", "Testing Accuracy"]
)

# Display the results table
depth_results_df
```

Out[38]:

|   | Max Depth | Training Accuracy | Testing Accuracy |
|---|-----------|-------------------|------------------|
| 0 | 1         | 0.871711          | 0.868421         |
| 1 | 2         | 0.888158          | 0.828947         |
| 2 | 3         | 0.904605          | 0.842105         |
| 3 | 4         | 0.957237          | 0.815789         |
| 4 | 5         | 0.980263          | 0.828947         |
| 5 | 6         | 0.993421          | 0.842105         |
| 6 | 7         | 0.996711          | 0.842105         |
| 7 | 8         | 1.000000          | 0.842105         |
| 8 | 9         | 1.000000          | 0.842105         |
| 9 | 10        | 1.000000          | 0.842105         |

```
In [39]: # Import matplotlib for visualization
import matplotlib.pyplot as plt

# Plot training accuracy for different tree depths
plt.plot(
    depth_results_df["Max Depth"],
    depth_results_df["Training Accuracy"],
    marker="o",
    label="Training Accuracy"
)

# Plot testing accuracy for different tree depths
plt.plot(
    depth_results_df["Max Depth"],
    depth_results_df["Testing Accuracy"],
    marker="o",
    label="Testing Accuracy"
)

# Add a title to the chart
plt.title("Overfitting and Underfitting Check")

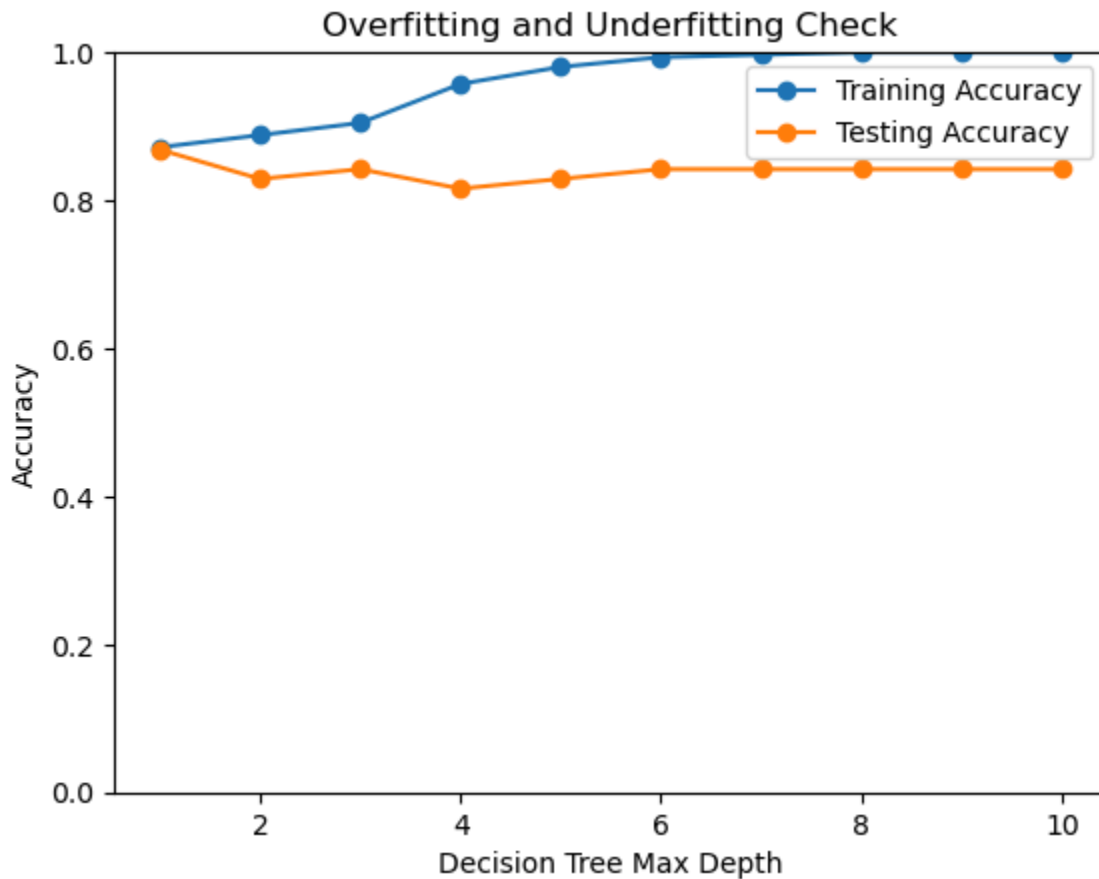
# Add a label for the x-axis
plt.xlabel("Decision Tree Max Depth")

# Add a label for the y-axis
plt.ylabel("Accuracy")

# Set the y-axis range from 0 to 1
plt.ylim(0, 1)

# Add a legend to explain the two lines
plt.legend()
```

```
# Show the chart  
plt.show()
```



## 10. Feature Importance

In this step, we check which features were most important for the Decision Tree model. Feature importance helps us understand which input variables had more influence on the model's predictions.

This is a simple explainability method. It helps us explain the model in a basic and business-friendly way.

```
In [40]: feature_importance_values = model.feature_importances_  
  
# Get the feature names from X_train  
feature_names = X_train.columns  
  
# Create a DataFrame to store feature names and their importance scores  
feature_importance_df = pd.DataFrame({  
    "Feature": feature_names,  
    "Importance": feature_importance_values  
})
```

```

# Sort the features from most important to least important
feature_importance_df = feature_importance_df.sort_values(
    by="Importance",
    ascending=False
)

# Display the feature importance table
feature_importance_df

```

Out[40]:

|    | Feature                        | Importance |
|----|--------------------------------|------------|
| 5  | emails_opened_last_month       | 0.244573   |
| 4  | website_visits_last_month      | 0.236271   |
| 2  | previous_purchases             | 0.231349   |
| 7  | days_since_last_purchase       | 0.147218   |
| 6  | ad_clicks_last_month           | 0.042591   |
| 12 | discount_received_Yes          | 0.041606   |
| 8  | customer_service_contacts      | 0.036889   |
| 1  | annual_income                  | 0.019503   |
| 0  | age                            | 0.000000   |
| 3  | average_order_value            | 0.000000   |
| 9  | membership_level_Gold          | 0.000000   |
| 10 | membership_level_Platinum      | 0.000000   |
| 11 | membership_level_Silver        | 0.000000   |
| 13 | preferred_channel_SMS          | 0.000000   |
| 14 | preferred_channel_Social Media | 0.000000   |
| 15 | preferred_channel_Website      | 0.000000   |

```

In [41]: # Select the top 10 most important features
top_features = feature_importance_df.head(10)

# Display the top 10 important features
top_features

```

Out[41]:

|    | Feature                   | Importance |
|----|---------------------------|------------|
| 5  | emails_opened_last_month  | 0.244573   |
| 4  | website_visits_last_month | 0.236271   |
| 2  | previous_purchases        | 0.231349   |
| 7  | days_since_last_purchase  | 0.147218   |
| 6  | ad_clicks_last_month      | 0.042591   |
| 12 | discount_received_Yes     | 0.041606   |
| 8  | customer_service_contacts | 0.036889   |
| 1  | annual_income             | 0.019503   |
| 0  | age                       | 0.000000   |
| 3  | average_order_value       | 0.000000   |

```
In [42]: # Import matplotlib for visualization
import matplotlib.pyplot as plt

# Create a horizontal bar chart for the top 10 important features
plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)

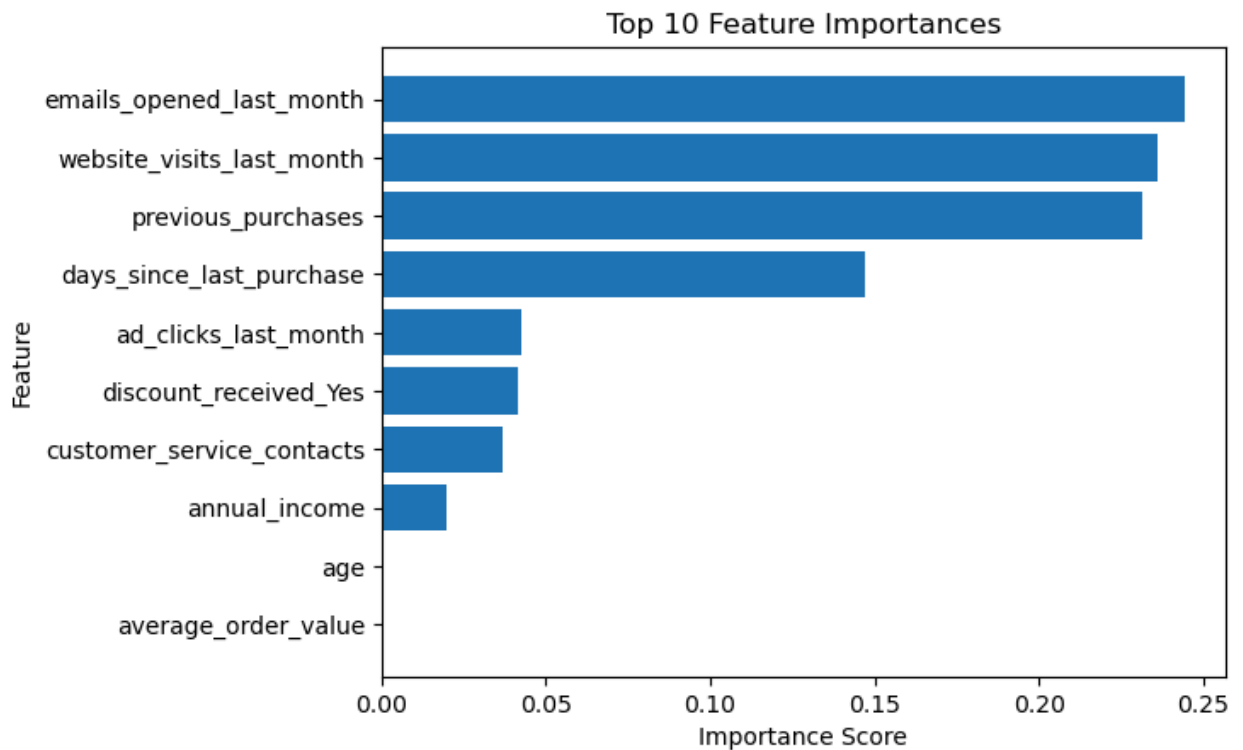
# Add a title to the chart
plt.title("Top 10 Feature Importances")

# Add a label for the x-axis
plt.xlabel("Importance Score")

# Add a label for the y-axis
plt.ylabel("Feature")

# Show the most important feature at the top
plt.gca().invert_yaxis()

# Show the chart
plt.show()
```



## 11. SHAP Demonstration

In this step, we use SHAP to explain the model predictions. SHAP helps us understand how each feature contributes to the model prediction.

Feature importance shows which features are important overall, but SHAP can explain both the overall model behavior and individual customer predictions.

In this notebook, we use SHAP in a simple way to explain the Decision Tree churn prediction model.

```
In [43]: # Run this cell only if you get an error that says: No module named 'shap'
!pip install shap
```

[illegible]

```

----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 1/2 [shap]
----- 2/2 [shap]

```

Successfully installed shap-0.52.0 slicer-0.0.8

```

In [44]: # Import SHAP for model explanation
import shap

# Import matplotlib for showing plots
import matplotlib.pyplot as plt

# Initialize SHAP JavaScript visualization for Jupyter Notebook
shap.initjs()

```



```

In [45]: # Create a SHAP explainer for the trained Decision Tree model
explainer = shap.TreeExplainer(model)

# Calculate SHAP values for the test data
shap_values = explainer.shap_values(X_test)

# Check the type of SHAP values
type(shap_values)

```

Out[45]: numpy.ndarray

```

In [46]: # Check whether shap_values is a list or an array
# Different SHAP versions may return the result in different formats
if isinstance(shap_values, list):

    # If SHAP returns a list, select index 1 for the churn class
    shap_values_class1 = shap_values[1]

    # Select the expected value for the churn class
    base_value_class1 = explainer.expected_value[1]

else:

    # If SHAP returns an array with three dimensions, select class 1
    shap_values_class1 = shap_values[:, :, 1]

    # Select the expected value for class 1
    base_value_class1 = explainer.expected_value[1]

```

```
# Display the shape of SHAP values for class 1  
shap_values_class1.shape
```

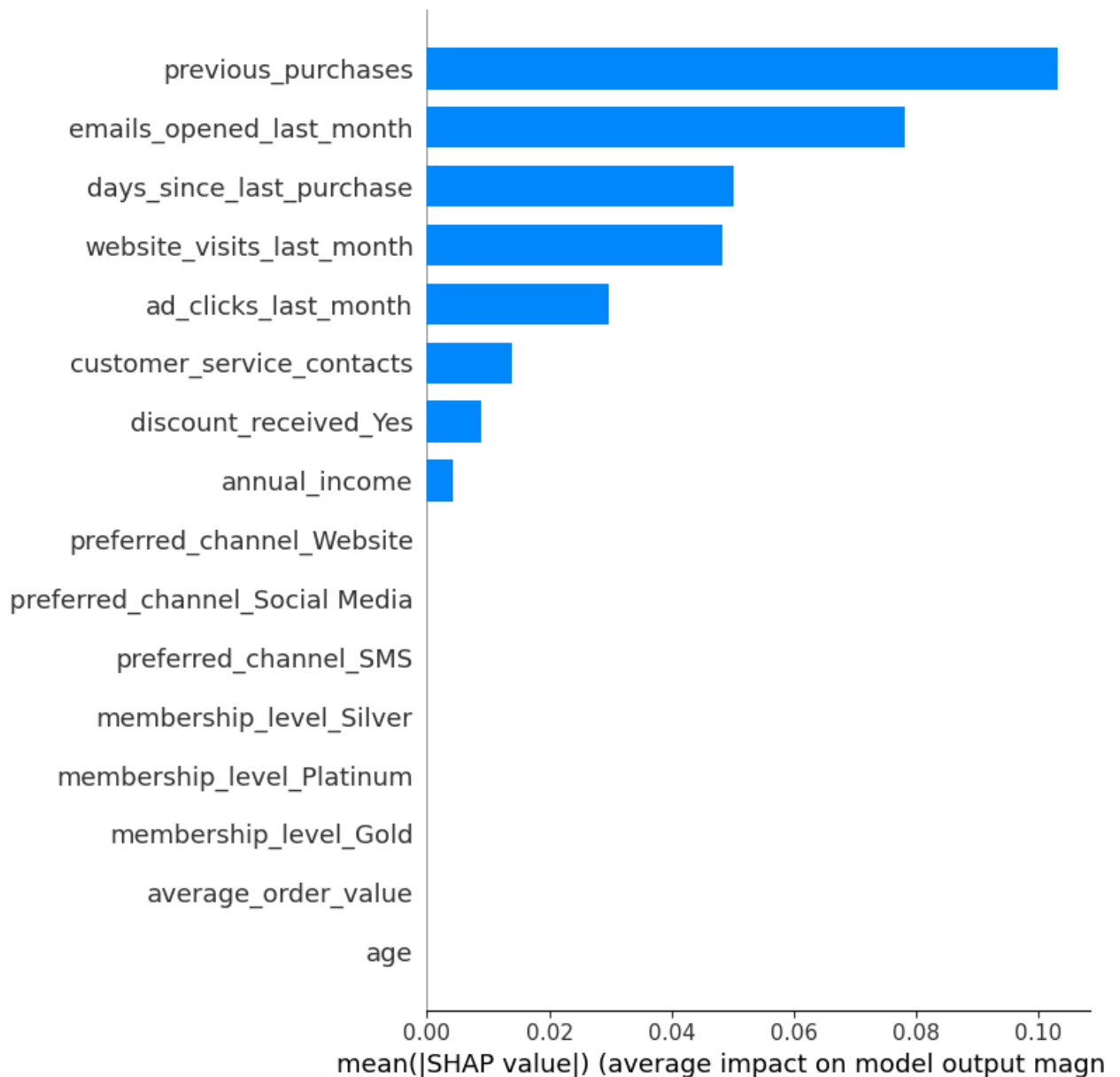
Out[46]: (76, 16)

Analysis of SHAP Values Shape The output (76, 16) shows the shape of the SHAP values for class 1, which represents Churn.

The first number 76, means that there are 76 customers in the test dataset. These are the customers that the model is explaining.

The second number 16, means that the model has 16 input features after preprocessing and one-hot encoding. For each customer, SHAP calculates one contribution value for each feature.

```
In [47]: # This plot shows the most important features based on SHAP values  
shap.summary_plot(  
    shap_values_class1,  
    X_test,  
    plot_type="bar"  
)
```



```
In [48]: # Select one customer from the test dataset
customer_index = 0

# Get the feature values for this customer
customer_data = X_test.iloc[customer_index]

# Get the SHAP values for this customer
customer_shap_values = shap_values_class1[customer_index]

# Create a DataFrame to show each feature and its SHAP contribution
customer_explanation = pd.DataFrame({
    "Feature": X_test.columns,
    "Feature Value": customer_data.values,
    "SHAP Value": customer_shap_values
})
```

```

# Create a column with absolute SHAP values to sort by importance
customer_explanation["Absolute SHAP Value"] = customer_explanation["SHAP Value"]

# Sort features by their absolute SHAP impact
customer_explanation = customer_explanation.sort_values(
    by="Absolute SHAP Value",
    ascending=False
)

# Display the explanation table for this customer
customer_explanation.head(10)

```

Out[48]:

|    | Feature                   | Feature Value | SHAP Value | Absolute SHAP Value |
|----|---------------------------|---------------|------------|---------------------|
| 2  | previous_purchases        | 21            | 0.075435   | 0.075435            |
| 6  | ad_clicks_last_month      | 7             | 0.039372   | 0.039372            |
| 8  | customer_service_contacts | 6             | -0.024289  | 0.024289            |
| 4  | website_visits_last_month | 34            | 0.014714   | 0.014714            |
| 7  | days_since_last_purchase  | 156           | 0.013789   | 0.013789            |
| 12 | discount_received_Yes     | True          | 0.002787   | 0.002787            |
| 1  | annual_income             | 34216         | 0.002344   | 0.002344            |
| 5  | emails_opened_last_month  | 6             | -0.002074  | 0.002074            |
| 0  | age                       | 60            | 0.000000   | 0.000000            |
| 3  | average_order_value       | 356.12        | 0.000000   | 0.000000            |

In [49]:

```

# Get the actual churn value for this customer
actual_value = y_test.iloc[customer_index]

# Get the predicted churn value for this customer
predicted_value = y_test_pred[customer_index]

# Create a small table to compare actual and predicted churn
single_customer_result = pd.DataFrame({
    "Actual Churn": [actual_value],
    "Predicted Churn": [predicted_value]
})

# Display the result
single_customer_result

```

Out[49]:

|   | Actual Churn | Predicted Churn |
|---|--------------|-----------------|
| 0 | 1            | 1               |

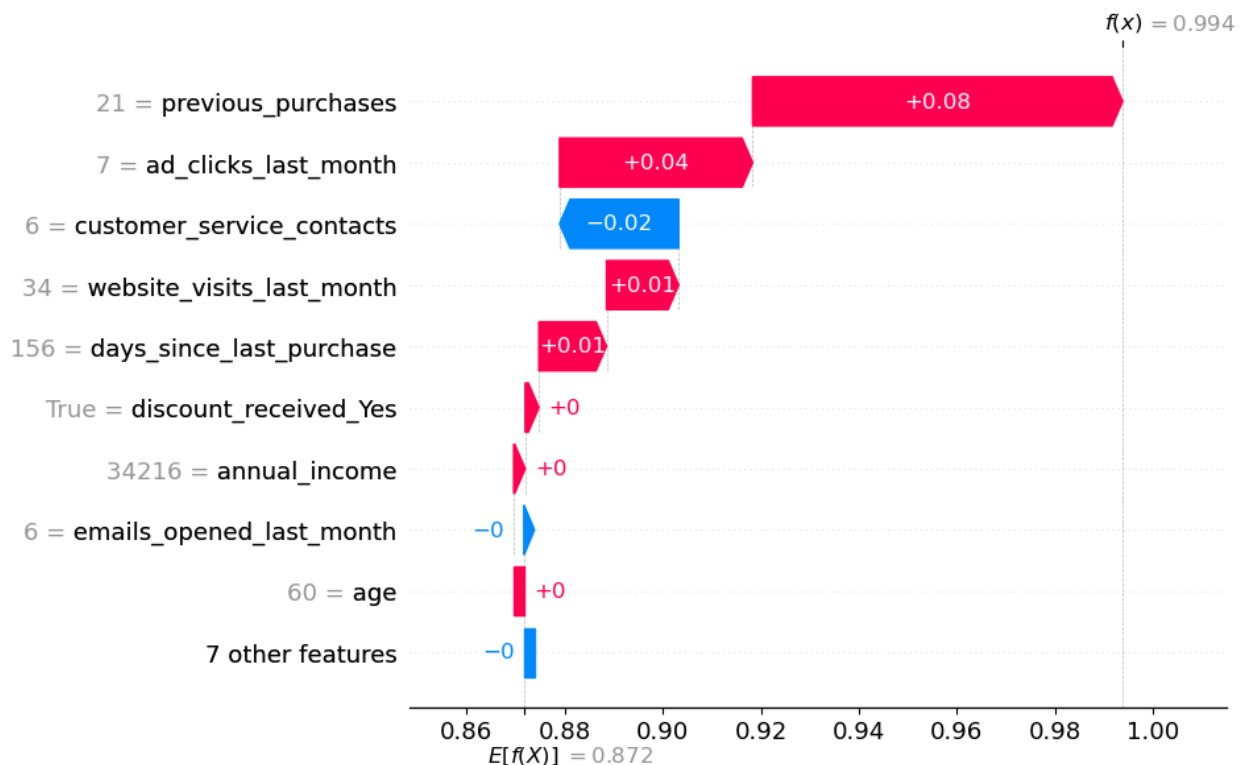
analysis of Single Customer Prediction This table compares the actual churn value

and the predicted churn value for one selected customer from the test dataset.

The Actual Churn value is 1, which means this customer actually is churn. The Predicted Churn value is also 1, which means the model predicted that this customer is churn.

```
In [50]: # Create a SHAP explanation object for one customer
single_customer_explanation = shap.Explanation(
    values=customer_shap_values,
    base_values=base_value_class1,
    data=customer_data,
    feature_names=X_test.columns
)

# Create a waterfall plot to explain this customer prediction
shap.plots.waterfall(single_customer_explanation)
```



## SHAP Waterfall Plot Analysis

This is a **SHAP (SHapley Additive exPlanations) waterfall plot** explaining an individual prediction from a machine learning model. Here's what it reveals:

# Overall Prediction

- **Base Value ( $E[f(X)]$ ):** 0.872 — the average model prediction across all data
- **Final Prediction ( $f(x)$ ):** 0.994 — the prediction for this specific instance
- **Net Change:** +0.122 (from base to final prediction)

The model is predicting a high probability (99.4%) for some binary outcome (likely customer purchase, retention, or conversion).

## Feature Contributions (Ranked by Impact)

### Positive Contributors (Push prediction UP)

1. **previous\_purchases (21):** +0.08 — **Strongest driver** of the high prediction
2. **ad\_clicks\_last\_month (7):** +0.04 — Moderate positive impact
3. **website\_visits\_last\_month (34):** +0.01 — Minor positive impact
4. **days\_since\_last\_purchase (156):** +0.01 — Minimal contribution
5. **discount\_received (True):** +0.00 — Essentially no impact
6. **annual\_income (34,216):** +0.00 — No effect
7. **age (60):** +0.00 — No effect

### Negative Contributors (Push prediction DOWN)

- **customer\_service\_contacts (6):** -0.02 — Only notable negative factor
- **emails\_opened\_last\_month (6):** -0.00 — Minimal negative impact
- **7 other features:** -0.00 — Negligible combined effect

## Key Insights

| Aspect            | Finding                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------|
| Primary Driver    | Historical purchase behavior (21 previous purchases) is by far the strongest predictor (+0.08) |
| Engagement Signal | Recent ad engagement (+0.04) and website visits (+0.01) reinforce the prediction               |

| Aspect                         | Finding                                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Risk Factor</b>             | Customer service contacts slightly reduce the prediction ( $-0.02$ ), possibly indicating unresolved issues |
| <b>Demographic Irrelevance</b> | Age, income, and discount status have zero predictive power for this customer                               |
| <b>Model Confidence</b>        | High prediction (0.994) suggests this is a straightforward case with strong purchase signals                |

## Business Interpretation

This customer is **highly likely to purchase** because they have a strong purchase history and recent engagement. The model considers this a "safe" prediction with minimal uncertainty, driven primarily by past behavior rather than demographic factors.

## 12. Simple LIME Demonstration

In this step, we use LIME to explain one individual prediction made by the model. LIME stands for Local Interpretable Model-agnostic Explanations.

LIME helps us understand why the model predicted Churn or Not Churn for one specific customer.

```
In [52]: # Run this cell only if you get an error that says: No module named 'lime'
!pip install lime
```

Defaulting to user installation because normal site-packages is not writeable  
Collecting lime  
 Downloading lime-0.2.0.1.tar.gz (275 kB)  
 Installing build dependencies: started  
 Installing build dependencies: finished with status 'done'  
 Getting requirements to build wheel: started  
 Getting requirements to build wheel: finished with status 'done'  
 Preparing metadata (pyproject.toml): started  
 Preparing metadata (pyproject.toml): finished with status 'done'  
Requirement already satisfied: matplotlib in d:\anaconda python\lib\site-packages (from lime) (3.10.6)  
Requirement already satisfied: numpy in d:\anaconda python\lib\site-packages (from lime) (2.3.5)  
Requirement already satisfied: scipy in d:\anaconda python\lib\site-packages (from lime) (1.16.3)  
Requirement already satisfied: tqdm in d:\anaconda python\lib\site-packages (from lime) (4.67.1)  
Requirement already satisfied: scikit-learn>=0.18 in d:\anaconda python\lib\site-packages (from lime) (1.7.2)  
Requirement already satisfied: scikit-image>=0.12 in d:\anaconda python\lib\site-packages (from lime) (0.25.2)  
Requirement already satisfied: networkx>=3.0 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (3.5)  
Requirement already satisfied: pillow>=10.1 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (12.0.0)  
Requirement already satisfied: imageio!=2.35.0,>=2.33 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (2.37.2)  
Requirement already satisfied: tifffile>=2022.8.12 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (2025.10.4)  
Requirement already satisfied: packaging>=21 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (25.0)  
Requirement already satisfied: lazy-loader>=0.4 in d:\anaconda python\lib\site-packages (from scikit-image>=0.12->lime) (0.4)  
Requirement already satisfied: joblib>=1.2.0 in d:\anaconda python\lib\site-packages (from scikit-learn>=0.18->lime) (1.5.2)  
Requirement already satisfied: threadpoolctl>=3.1.0 in d:\anaconda python\lib\site-packages (from scikit-learn>=0.18->lime) (3.5.0)  
Requirement already satisfied: contourpy>=1.0.1 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (1.3.3)  
Requirement already satisfied: cycler>=0.10 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (0.11.0)  
Requirement already satisfied: fonttools>=4.22.0 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (4.60.1)  
Requirement already satisfied: kiwisolver>=1.3.1 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (1.4.9)  
Requirement already satisfied: pyparsing>=2.3.1 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (3.2.5)  
Requirement already satisfied: python-dateutil>=2.7 in d:\anaconda python\lib\site-packages (from matplotlib->lime) (2.9.0.post0)  
Requirement already satisfied: six>=1.5 in d:\anaconda python\lib\site-packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)  
Requirement already satisfied: colorama in d:\anaconda python\lib\site-packages (from tqdm->lime) (0.4.6)  
Building wheels for collected packages: lime

```
Building wheel for lime (pyproject.toml): started
Building wheel for lime (pyproject.toml): finished with status 'done'
Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283917 sha256=85782ff1293cb09244f1e1ccaac66c901e031bf0eb746d3a9e2903801da4075b
Stored in directory: c:\users\sai lalit\appdata\local\pip\cache\wheels\7c\04\5c\157dc9106512a6c7a30653ec064490c94a49e0fc8f63d19ab9
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

```
In [53]: # Import numpy for numerical operations
import numpy as np

# Import pandas for creating tables
import pandas as pd

# Import matplotlib for visualization
import matplotlib.pyplot as plt

# Import LimeTabularExplainer for explaining tabular data models
from lime.lime_tabular import LimeTabularExplainer
```

```
In [54]: # Convert X_train to float format for LIME
X_train_lime = X_train.astype(float)

# Convert X_test to float format for LIME
X_test_lime = X_test.astype(float)

# Store the feature names
feature_names = X_train_lime.columns.tolist()

# Define class names for the target variable
class_names = ["Not Churn", "Churn"]
```

```
In [55]: # Create a LIME explainer for tabular data
lime_explainer = LimeTabularExplainer(

    # Use the training data as the background data for LIME
    training_data=X_train_lime.values,

    # Provide the feature names
    feature_names=feature_names,

    # Provide the class names
    class_names=class_names,

    # Set the mode to classification because churn is a classification problem
    mode="classification",

    # Discretize continuous variables to make explanations easier to understand
    discretize_continuous=True
)
```

```
In [56]: # Select one customer from the test data
customer_index = 0

# Get the selected customer's feature values
customer_instance = X_test_lime.iloc[customer_index]

# Display the selected customer data
customer_instance
```

```
Out[56]: age                60.00
annual_income            34216.00
previous_purchases        21.00
average_order_value       356.12
website_visits_last_month  34.00
emails_opened_last_month   6.00
ad_clicks_last_month       7.00
days_since_last_purchase  156.00
customer_service_contacts   6.00
membership_level_Gold       0.00
membership_level_Platinum   0.00
membership_level_Silver     0.00
discount_received_Yes       1.00
preferred_channel_SMS       1.00
preferred_channel_Social Media 0.00
preferred_channel_Website   0.00
Name: 361, dtype: float64
```

```
In [57]: # Generate a LIME explanation for the selected customer
lime_explanation = lime_explainer.explain_instance(

    # The selected customer data
    data_row=customer_instance.values,

    # The model prediction probability function
    predict_fn=model.predict_proba,

    # Show the top 10 most important features for this explanation
    num_features=10
)
```

```
D:\anaconda python\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

```
In [58]: # Get the actual churn value for this customer
actual_value = y_test.iloc[customer_index]

# Get the predicted churn value for this customer
predicted_value = model.predict(X_test_lime.iloc[[customer_index]])[0]

# Get the predicted probabilities for Not Churn and Churn
predicted_probabilities = model.predict_proba(X_test_lime.iloc[[customer_index]
```

```
# Create a small table to show the result
lime_prediction_result = pd.DataFrame({
    "Actual Churn": [actual_value],
    "Predicted Churn": [predicted_value],
    "Probability of Not Churn": [predicted_probabilities[0]],
    "Probability of Churn": [predicted_probabilities[1]]
})

# Display the result
lime_prediction_result
```

```
Out[58]:
```

|          | <b>Actual Churn</b> | <b>Predicted Churn</b> | <b>Probability of Not Churn</b> | <b>Probability of Churn</b> |
|----------|---------------------|------------------------|---------------------------------|-----------------------------|
| <b>0</b> | 1                   | 1                      | 0.006211                        | 0.993789                    |

```
In [59]: # Convert the LIME explanation into a readable table
lime_explanation_table = pd.DataFrame(
    lime_explanation.as_list(),
    columns=["Feature Condition", "Contribution"]
)

# Display the LIME explanation table
lime_explanation_table
```

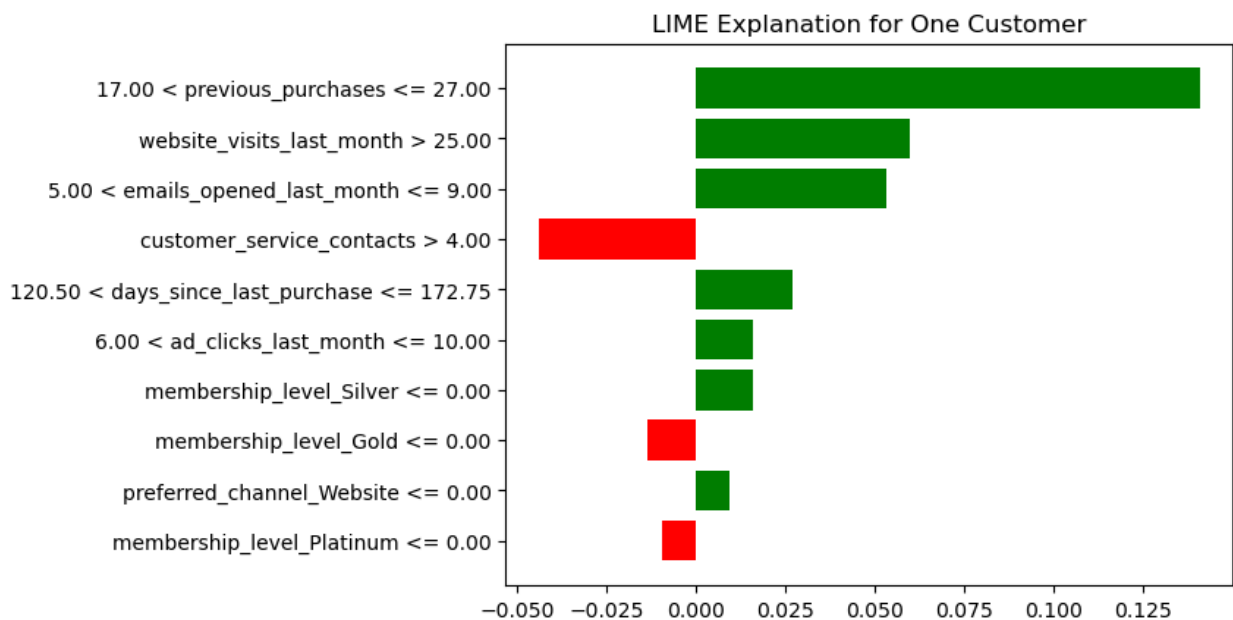
```
Out[59]:
```

|          | <b>Feature Condition</b>                    | <b>Contribution</b> |
|----------|---------------------------------------------|---------------------|
| <b>0</b> | 17.00 < previous_purchases <= 27.00         | 0.140702            |
| <b>1</b> | website_visits_last_month > 25.00           | 0.059838            |
| <b>2</b> | 5.00 < emails_opened_last_month <= 9.00     | 0.053217            |
| <b>3</b> | customer_service_contacts > 4.00            | -0.043912           |
| <b>4</b> | 120.50 < days_since_last_purchase <= 172.75 | 0.027018            |
| <b>5</b> | 6.00 < ad_clicks_last_month <= 10.00        | 0.015838            |
| <b>6</b> | membership_level_Silver <= 0.00             | 0.015751            |
| <b>7</b> | membership_level_Gold <= 0.00               | -0.013749           |
| <b>8</b> | preferred_channel_Website <= 0.00           | 0.009559            |
| <b>9</b> | membership_level_Platinum <= 0.00           | -0.009440           |

```
In [60]: # Create a LIME explanation chart
fig = lime_explanation.as_pyplot_figure()

# Add a title to the chart
plt.title("LIME Explanation for One Customer")

# Show the chart
plt.show()
```



```
In [61]: # Import display and HTML from the correct IPython display module
from IPython.display import display, HTML

# Convert the LIME explanation to HTML format
lime_html = lime_explanation.as_html()

# Display the LIME explanation as HTML inside the notebook
display(HTML(lime_html))
```

## 13. Fairness and Bias Reflection

In this step, we reflect on fairness and bias in the machine learning model. Fairness means checking whether the model behaves differently for different customer groups.

In this notebook, we look at group columns such as gender, age\_group, and region. These columns were not used directly as model input, but we can still use them to check whether the model results are different across groups.

```
In [62]: # Use the trained model to predict churn values for the test data
y_test_pred = model.predict(X_test)

# Select fairness-related columns for the same customers in the test data
fairness_data = df.loc[y_test.index, ["customer_id", "gender", "age_group", "r

# Add the actual churn values to the fairness table
fairness_data["Actual Churn"] = y_test.values

# Add the predicted churn values to the fairness table
```

```

fairness_data["Predicted Churn"] = y_test_pred

# Create a column to show whether each prediction is correct
fairness_data["Correct Prediction"] = fairness_data["Actual Churn"] == fairness_data["Predicted Churn"]

# Display the first 10 rows
fairness_data.head(10)

```

Out[62]:

|            | customer_id | gender | age_group | region | Actual Churn | Predicted Churn | Correct Prediction |
|------------|-------------|--------|-----------|--------|--------------|-----------------|--------------------|
| <b>361</b> | MC0362      | Male   | 51+       | North  | 1            | 1               | True               |
| <b>48</b>  | MC0049      | Male   | 51+       | West   | 1            | 1               | True               |
| <b>35</b>  | MC0036      | Male   | 51+       | West   | 1            | 0               | False              |
| <b>54</b>  | MC0055      | Female | 36-50     | North  | 1            | 1               | True               |
| <b>303</b> | MC0304      | Female | 26-35     | East   | 1            | 1               | True               |
| <b>28</b>  | MC0029      | Male   | 26-35     | West   | 1            | 1               | True               |
| <b>87</b>  | MC0088      | Female | 36-50     | North  | 1            | 1               | True               |
| <b>229</b> | MC0230      | Female | 51+       | West   | 1            | 1               | True               |
| <b>175</b> | MC0176      | Male   | 51+       | East   | 1            | 1               | True               |
| <b>146</b> | MC0147      | Male   | 51+       | South  | 1            | 1               | True               |

In [63]: *# Define a function to summarize model performance by group*  
**def** fairness\_summary(group\_column):

```

    # Group the fairness data by the selected group column
    summary = fairness_data.groupby(group_column).agg(

        # Count how many customers exist in each group
        Number_of_Customers=("customer_id", "count"),

        # Calculate the actual churn rate in each group
        Actual_Churn_Rate=("Actual Churn", "mean"),

        # Calculate the predicted churn rate in each group
        Predicted_Churn_Rate=("Predicted Churn", "mean"),

        # Calculate the accuracy in each group
        Accuracy=("Correct Prediction", "mean")
    ).reset_index()

    # Return the summary table
    return summary

```

In [64]: *# Create a fairness summary by gender*  
gender\_fairness = fairness\_summary("gender")

```
# Display the fairness summary by gender
gender_fairness
```

```
Out[64]:
```

|   | gender | Number_of_Customers | Actual_Churn_Rate | Predicted_Churn_Rate | Acc |
|---|--------|---------------------|-------------------|----------------------|-----|
| 0 | Female | 43                  | 0.860465          | 0.953488             | 0.8 |
| 1 | Male   | 33                  | 0.878788          | 0.878788             | 0.8 |

```
In [65]: # Create a fairness summary by age group
age_group_fairness = fairness_summary("age_group")

# Display the fairness summary by age group
age_group_fairness
```

```
Out[65]:
```

|   | age_group | Number_of_Customers | Actual_Churn_Rate | Predicted_Churn_Rate |
|---|-----------|---------------------|-------------------|----------------------|
| 0 | 18-25     | 11                  | 1.000000          | 1.000000             |
| 1 | 26-35     | 13                  | 0.769231          | 0.923077             |
| 2 | 36-50     | 16                  | 0.937500          | 0.937500             |
| 3 | 51+       | 36                  | 0.833333          | 0.888889             |

```
In [66]: # Create a fairness summary by region
region_fairness = fairness_summary("region")

# Display the fairness summary by region
region_fairness
```

```
Out[66]:
```

|   | region | Number_of_Customers | Actual_Churn_Rate | Predicted_Churn_Rate | Acc |
|---|--------|---------------------|-------------------|----------------------|-----|
| 0 | East   | 21                  | 0.857143          | 0.952381             | 0.8 |
| 1 | North  | 19                  | 0.736842          | 0.894737             | 0.7 |
| 2 | South  | 13                  | 0.846154          | 0.846154             | 0.6 |
| 3 | West   | 23                  | 1.000000          | 0.956522             | 0.9 |

## How to Read the Fairness Tables

**Number of Customers** shows how many test customers are in each group. If one group has very few customers, the result may not be reliable.

**Actual Churn Rate** shows the real churn rate for each group. This tells us how many customers actually churned in that group.

**Predicted Churn Rate** shows how often the model predicted churn for each group. If this value is much higher for one group, the model may be predicting

churn more often for that group.

**Accuracy** shows how often the model was correct for each group. If accuracy is much lower for one group, the model may not work equally well for all groups.

```
In [67]: # Create a simple bar chart for predicted churn rate by gender
gender_fairness.plot(
    x="gender",
    y="Predicted_Churn_Rate",
    kind="bar",
    legend=False
)

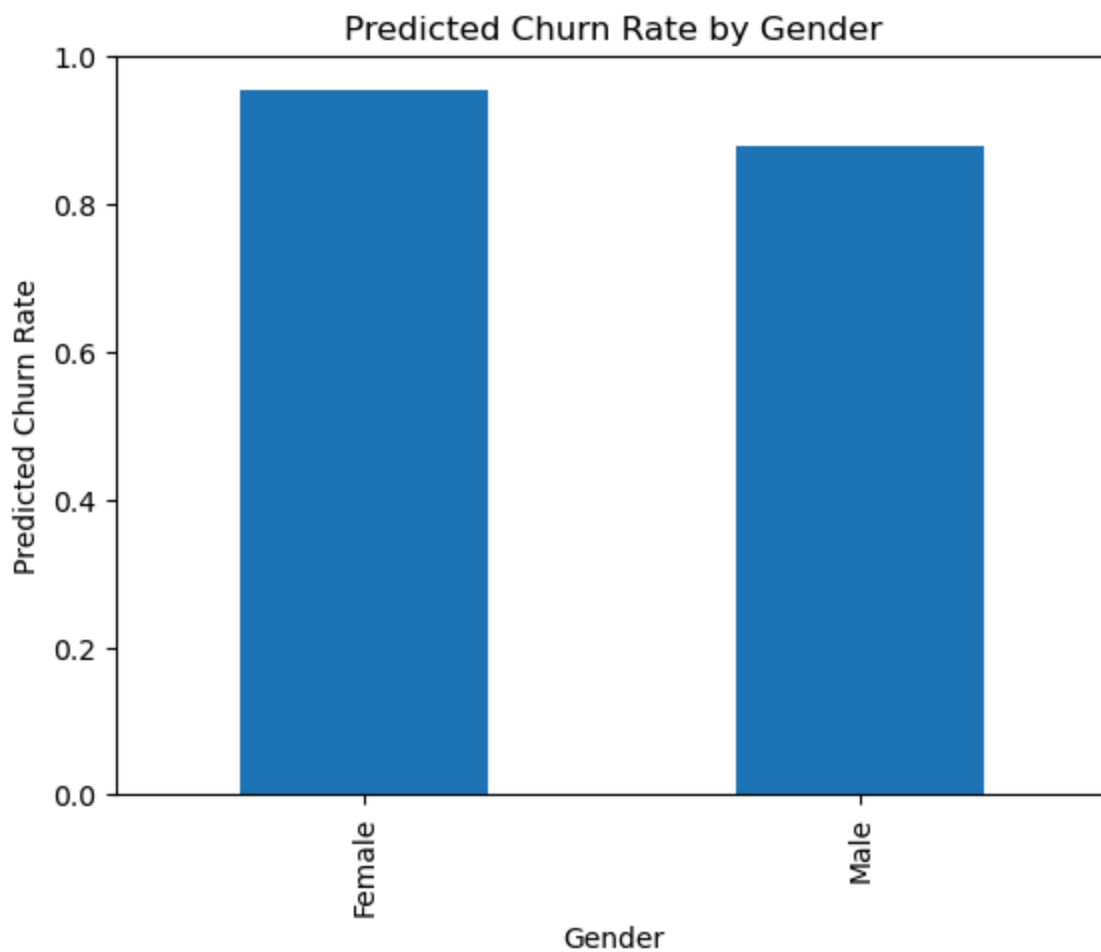
# Add a title to the chart
plt.title("Predicted Churn Rate by Gender")

# Add a label for the x-axis
plt.xlabel("Gender")

# Add a label for the y-axis
plt.ylabel("Predicted Churn Rate")

# Set y-axis from 0 to 1
plt.ylim(0, 1)

# Show the chart
plt.show()
```



```
In [68]: # Create a simple bar chart for accuracy by gender
gender_fairness.plot(
    x="gender",
    y="Accuracy",
    kind="bar",
    legend=False
)

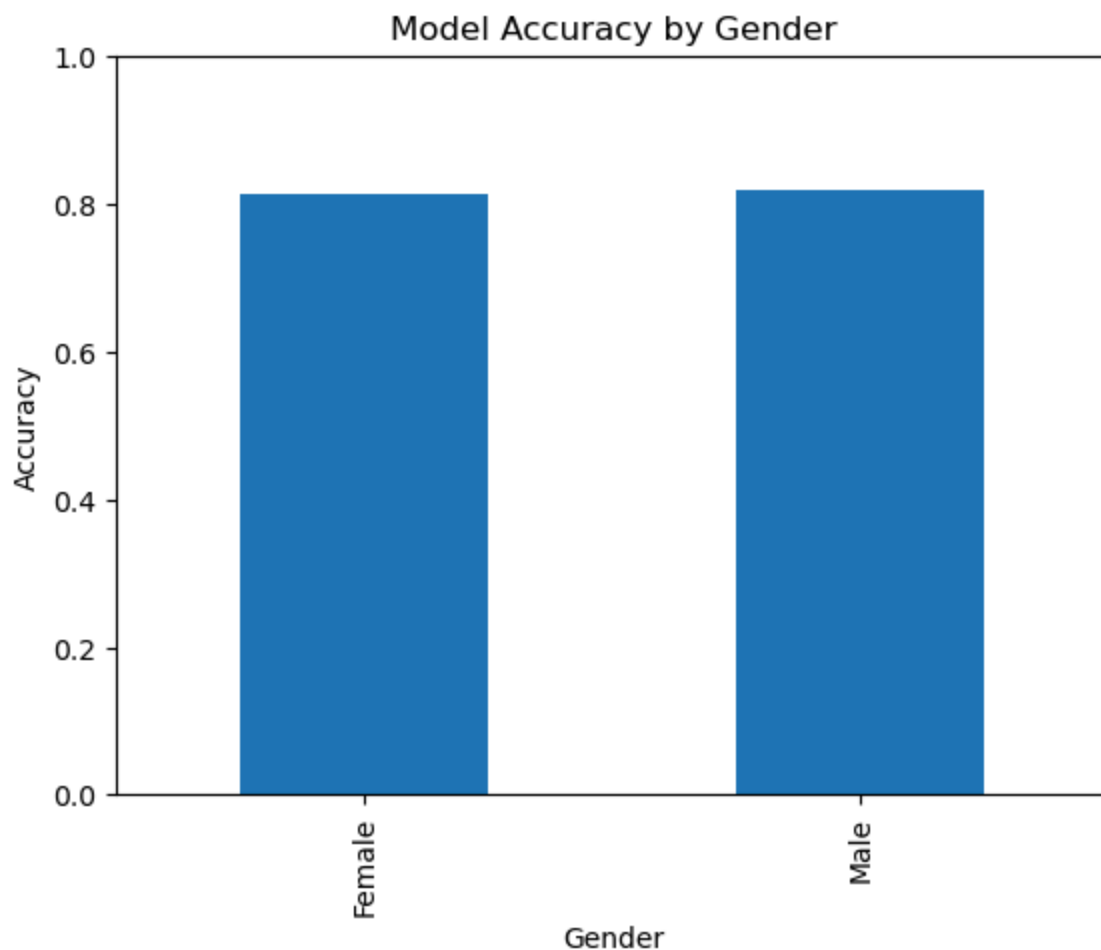
# Add a title to the chart
plt.title("Model Accuracy by Gender")

# Add a label for the x-axis
plt.xlabel("Gender")

# Add a label for the y-axis
plt.ylabel("Accuracy")

# Set y-axis from 0 to 1
plt.ylim(0, 1)

# Show the chart
plt.show()
```



### **Fairness and Bias Reflection**

The fairness tables help us compare model behavior across different customer groups. We can check whether the model predicts churn more often for one group than another group. We can also check whether the model accuracy is lower for some groups.

If one group has a much higher predicted churn rate than other groups, we should ask whether this difference is reasonable or whether it may show bias in the data or model.

If one group has much lower accuracy, the model may be less reliable for that group. This can create unfair or unequal business decisions.

## **14. Business Interpretation**

In this final step, we interpret the machine learning results from a business perspective. The goal is not only to evaluate the model technically, but also to

understand what the results mean for customer churn management.

```
In [69]: # Import required evaluation metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Predict churn values for training data
y_train_pred = model.predict(X_train)

# Predict churn values for testing data
y_test_pred = model.predict(X_test)

# Calculate training accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Calculate testing accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

# Calculate precision
precision = precision_score(y_test, y_test_pred, zero_division=0)

# Calculate recall
recall = recall_score(y_test, y_test_pred, zero_division=0)

# Calculate F1-score
f1 = f1_score(y_test, y_test_pred, zero_division=0)

# Create confusion matrix
cm = confusion_matrix(y_test, y_test_pred)

# Extract True Negative, False Positive, False Negative, and True Positive
tn, fp, fn, tp = cm.ravel()

# Create a final business summary table
business_summary = pd.DataFrame({
    "Result": [
        "Training Accuracy",
        "Testing Accuracy",
        "Precision",
        "Recall",
        "F1-Score",
        "True Negative",
        "False Positive",
        "False Negative",
        "True Positive"],
    "Value": [
        train_accuracy,
        test_accuracy,
        precision,
        recall,
        f1,
        tn,
        fp,
        fn,
        tp]
```

```

        fn,
        tp],

    "Business Meaning": [
        "How well the model learned from old data",
        "How well the model works on new customers",
        "How many predicted churn customers were actually churn customers",
        "How many actual churn customers were correctly identified",
        "Balance between precision and recall",
        "Customers correctly predicted as not churn",
        "Customers wrongly predicted as churn",
        "Churn customers missed by the model",
        "Churn customers correctly identified"]
})

# Display the final business summary
business_summary

```

Out[69]:

|   | Result            | Value     | Business Meaning                                  |
|---|-------------------|-----------|---------------------------------------------------|
| 0 | Training Accuracy | 0.957237  | How well the model learned from old data          |
| 1 | Testing Accuracy  | 0.815789  | How well the model works on new customers         |
| 2 | Precision         | 0.871429  | How many predicted churn customers were actual... |
| 3 | Recall            | 0.924242  | How many actual churn customers were correctly... |
| 4 | F1-Score          | 0.897059  | Balance between precision and recall              |
| 5 | True Negative     | 1.000000  | Customers correctly predicted as not churn        |
| 6 | False Positive    | 9.000000  | Customers wrongly predicted as churn              |
| 7 | False Negative    | 5.000000  | Churn customers missed by the model               |
| 8 | True Positive     | 61.000000 | Churn customers correctly identified              |

| Metric                   | Value  | Business Interpretation                                                                                                                                       |
|--------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Training Accuracy</b> | 95.72% | The model learned patterns from historical customer data very well, indicating strong performance on the training dataset.                                    |
| <b>Testing Accuracy</b>  | 81.58% | The model correctly predicts customer churn for approximately 82 out of every 100 new customers, showing good real-world predictive capability.               |
| <b>Precision</b>         | 87.14% | When the model predicts that a customer will churn, it is correct about 87% of the time. This helps reduce unnecessary retention efforts and marketing costs. |
| <b>Recall</b>            | 92.42% | The model successfully identifies over 92% of actual churn customers, ensuring that most at-risk customers are detected before they leave.                    |
| <b>F1-Score</b>          | 89.71% | The model maintains a strong balance between identifying churn customers and minimizing incorrect churn predictions, making it                                |

| Metric                 | Value | Business Interpretation                                                                                                                                    |
|------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        |       | reliable for business decision-making.                                                                                                                     |
| <b>True Positives</b>  | 61    | The model correctly identified 61 customers who were likely to churn, allowing the company to target them with retention strategies.                       |
| <b>False Positives</b> | 9     | Nine customers were incorrectly predicted to churn. While these customers may receive unnecessary retention offers, the business impact is relatively low. |
| <b>False Negatives</b> | 5     | Only five actual churn customers were missed by the model. These customers may leave without intervention, representing a small potential revenue loss.    |
| <b>True Negatives</b>  | 1     | One customer was correctly identified as not likely to churn, demonstrating the model's ability to recognize loyal customers.                              |

## Overall Business Conclusion

From the analysis, it can be concluded that the churn prediction model is appropriate to deploy as an application to assist in retaining the customers. The testing accuracy of the model was at 81.58%, precision at 87.14%, recall at 92.42% and F1 score at 89.71%. All these scores indicate high levels of predictive capability of the model. The high recall rate makes the model quite significant because it guarantees that most of the customers who are going to churn are correctly predicted.

In this case, the model managed to predict 61 customers who will churn and only five customers were not detected as churn risks. Even though nine customers who will not churn were considered churn risks, the cost of implementing retention measures to such customers is relatively lower compared to the loss associated with the actual customers.

the model is highly valuable in the business context as it enables early detection of churn risks and assists in making decisions on customer retention.

In [ ]: